

Reinforcement Learning

Classical RL Algorithms on Discrete and Continuous MDPs

Michael Peluso

November 23, 2025

CS7641: Machine Learning Fall 2025

Professor Theodore J. LaGrow

GitHub Repository

Abstract—Reinforcement learning algorithms divide into model-based methods, which construct explicit transition models for dynamic-programming solutions, and model-free methods, which learn action values through temporal-difference updates from delayed rewards. We apply Value Iteration (VI) and Policy Iteration (PI) alongside SARSA (on-policy) and Q-Learning (off-policy) to two contrasting Markov Decision Problems: the stochastic, discrete Blackjack environment and the deterministic, continuous CartPole simulation. In Blackjack, model-based methods excel with few states, with VI and PI convergence outperforming SARSA and Q-Learning, whereas model-free methods achieve stronger returns in solving CartPole by focusing updates on experienced states rather than sweeping coarse discretization. SARSA displays tighter variance under its conservative on-policy among Blackjack’s stochasticity, while Q-Learning’s greedy maximization encourages aggressive exploration in CartPole. These results confirm that algorithm selection should prioritize state-space structure and transition-model availability over raw computational cost.

INTRODUCTION

Reinforcement learning provides an enhanced framework for generating predictions in environments that offer delayed rewards, making it particularly suited for tasks with significant long-term consequences. It supplies agents with optimal policies through trial-and-error interactions, yielding a balance between exploration and exploitation. Resembling conventional dynamic programming algorithms, reinforcement learning is applied to tasks where the environmental model is unknown, and it restricts learning through interaction and evaluation feedback.

Objective

This report analyzes the performance of each practice across two contrasting Markov Decision Processes(MDP), exposing how each method behaves under uncertainty and sequential decision-making constraints. Measuring the performance of model-based dynamic programming(DP) using Value Iteration(VI) and Policy Iteration(PI) coupled with SARSA and Q-Learning’s model-free reinforcement learning(RL) allows us to quantify differing features and distinguish how each compares in different canonical environments. The algorithms are first tested in a Blackjack environment to decide between hitting or standing, and then are tasked with balancing a standing pole fastened to a one-dimensional moving cart. Metrics such as convergence speed, computational cost, and policy quality are used to identify the outperforming algorithms in both discrete and continuous settings while also displaying the impacts of and sensitivity to stochasticity and state-space discretization. We calculate which algorithm paradigm is most efficient and robust for each MDP type and highlight the tradeoff between them.

Hypotheses

Blackjack Learning: The turn-based card game is inherently stochastic by systematically implementing randomness into each card draw, creating a sense of chance through shuffling and a hidden dealer card. These probabilities can be statistically solved with enough understanding of the game, so VI and PI will leverage their environmental knowledge to outperform model-free methods in both convergence speed and policy quality. The models are equipped with the direct house rules to compute probabilities and memorize, while SARSA and Q-Learning must reverse-engineer a rule set skewed by random chance. Due to the limited discrete state space, VI and PI should converge within a few iterations with nearly optimal strategies, while SARSA and Q-Learning must average a multitude of noisy probability spaces and delayed rewards to approximate the same policy [1]. The model-free methods should result in slower learning with higher variance.

CartPole Learning: Alternatively, the physics-based CartPole simulation is entirely deterministic across its continuous states. SARSA and Q-Learning will focus only on commonly experienced states, avoiding unreachable corners of the grid while refining policies with upright pole trajectories repeatedly and receiving rewarding feedback on the meaningful states. Meanwhile, model-based methods require splicing the pole’s angles and angular velocities into discrete bins, distorting its continuous nature into a coarse, pixilated grid [1]. VI and PI then sweep through each blurred state, measuring the unreachable states as often as the common ones. Therefore, the model-free methods should clearly show faster convergence and efficiency, resulting in better quality policies.

On-Policy and Off-Policy: Q-Learning’s greedy maximization inflates policies regardless of what action is taken next, demonstrating vulnerabilities to noisy environments with outlying samples. More intuitively, this approach teaches the agent to gamble greedily after experiencing a rare hand of blackjack. SARSA, by contrast, updates its policies regarding the actions it takes next, incorporating the cost of exploration directly into its estimates. The on-policy’s conservative search should then result in more stable policies in stochastic environments.

In the CartPole environment, Q-Learning will have an edge over SARSA by exploring riskier states without stochastic uncertainty or randomness. The on-policy method naturally exploits the safe, repeatable trajectories it experiences, while Q-Learning’s off-policy approach uses deterministic physics to lean towards the truly best action more aggressively [2]. While SARSA’s restrictive exploitation will outperform Q-Learning with Blackjack’s

randomness, it will fail to deliver a more efficient convergence when run on CartPole.

METHODOLOGY

This section outlines the experimental framework used to solve both Markov Decision Problems using model-based and model-free reinforcement algorithms. Blackjack and Cartpole settings from Gymnasium serve as contrasting environments, integrating either stochastic transitions or deterministic physics within their respective state spaces. Each algorithm applies validated hyperparameters through a unified entry point file and YAML configuration, capturing performance metric variance for clear visual reports.

Environments

Blackjack: The classic turn-based card game involves the player competing with a dealer to achieve a hand of playing cards whose sum is greater than the opponent’s but does not exceed twenty-one. The player receives two cards at the start of the round, holding a sum between four and twenty-one, where an ace is measured as either one or eleven at the holder’s discretion. Only one of the opponent’s cards is visible, forcing the player to use probabilistic guesses to reach a competitive sum by “standing” at their current sum or “hitting” for another card without “busting” over twenty-one.

The Gym Blackjack-v1 environment [3] emulates these odds, containing about 280 discrete states derived from the combinations of possible player sums, including an ace, and the card shown by the dealer. The agent repeatedly decides between drawing another card, risking a bust, or standing to allow the dealer to play their turn. The opponent must hit on all hands that sum below seventeen, only hitting on a “soft” seventeen when holding an ace that is counted as one. The player will lose one point from a bust or an opponent win, but may only be awarded a point by achieving a sum greater than the dealer’s sum, which does not exceed twenty-one. A draw has no effect on score. The agent’s objective is to maximize wins across episodes, but because the environment favors the dealer, we consider breaking even a successful outcome.

The very nature of the hidden cards introduces chance and stochasticity into the game, where the probabilities of the next draw can be calculated, but the results are never known. This simulation prevents card counting by drawing from an infinite deck, substantially minimizing the information required in calculations. In other words, sampling cards with instant replacement generates a stationary draw probability and supports a fully Markovian environment where history does not affect future state transitions.

Cart Pole: This environment, implemented by Gymnasium CartPole-v1 [3], simulates a two-dimensional scene containing a square cart that can roll either left or right, with an upright pole anchored to its top side that can also tilt left or right. The agent’s goal is to keep the pole balanced by pushing the cart left or right, earning a reward for every timestep the pole stays within twelve degrees of being vertical and the cart remains within the five-unit track limits. We consider this MDP solved when the agent can successfully last 195 steps within the bounds for one hundred consecutive episodes.

The obvious difference between this and the Blackjack environment, within the context of Markovian Decision Problems, is that the physics simulation naturally generates a four-dimensional

continuous state space consisting of the pole’s leaning angle and angular velocity, coupled with the cart’s horizontal position and horizontal velocity. However, our model-free algorithms operate on a discretized state space to index the Q-table, while the model-based methods require a fully discrete and known transition model. Therefore, we clamped the position, angle, and velocity bounds and used coarse bin ranges to generate between eight hundred and two thousand total states across all four models. Discretization induces state aliasing by mapping ranges of consecutive states to discrete bins, preventing infinite state resolution while reducing precision. Model-based methods use finer additional bins, offering a more competitive advantage given their tendency to ignore the importance of temporal state. For our environment, pole angles and angular velocity have stricter limits and are more critical to the reward, so finer bins were used for those parameters.

TABLE I
CARTPOLE DISCRETIZATION IMPACT

Bin Config	Total States	Mean Return	Wall-Clock (s)
<i>Model-Free (SARSA/Q-Learning)</i>			
[3, 3, 6, 8]	432	185.2	142.3
[3, 3, 8, 12]	864	229.1	189.5
[4, 4, 10, 12]	1,920	241.6	267.8
<i>Model-Based (VI/PI)</i>			
[3, 3, 8, 12]	864	38.4	12.1
[4, 4, 10, 12]	1,920	52.9	32.2
[5, 5, 12, 14]	4,200	61.3	187.4

Algorithms

We experiment with four tabular learning methods, with the key distinction being whether each method relies on a transition model of the environment. Model-based methods first explicitly learn the full transition model and reward function. By essentially building a map of their environment, the agent iteratively solves the Bellman equation for each state until convergence, achieving the optimal policies for the MDP with very few environmental interactions.

Model-free algorithms simply explore their environment, using ϵ -greedy policies with tabular Q-values initialized to zero. A linearly decaying randomness multiplier of one facilitates greedy action, ensuring early exploration of unseen states despite sparse environmental rewards. They learn the relationship between actions and rewards directly from sampled experiences using temporal-difference updates, meaning future state-action values are measured based on their greedy returns. This approach is

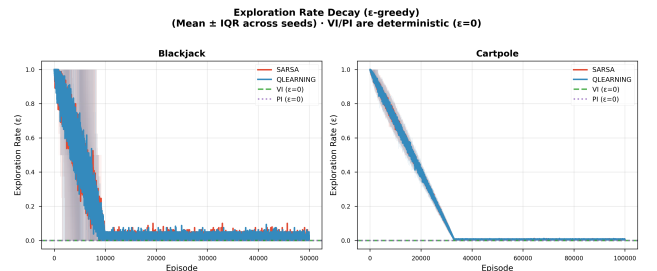


Fig. 1. Exploration schedule (ϵ -greedy decay). Blackjack decays over 9,927 episodes; CartPole uses extended decay over 33,000 episodes for broader state coverage.

TABLE II
HYPERPARAMETER CONFIGURATION AND VALIDATION SUMMARY

Algorithm	Parameter	Tested Range	Final Value	Justification
<i>Model-Free Methods (SARSA & Q-Learning)</i>				
Shared	Learning rate α	[0.01, 0.20]	0.037	Stable updates; damps variance
Shared	Discount factor γ	[0.90, 0.999]	0.9709	Balances short/long-horizon rewards
Shared	Initial ϵ	1.0 (fixed)	1.0	Full exploration at start
Blackjack	ϵ -decay episodes	[5,000, 20,000]	9,927	~20% of 50k total episodes
Blackjack	ϵ -floor	[0.001, 0.02]	0.0061	Minimal residual exploration
Blackjack	Training episodes	—	50,000	Convergence at ~6k; margin for refinement
CartPole	ϵ -decay episodes	[10,000, 50,000]	33,000	~33% of 100k; extended state coverage
CartPole	ϵ -floor	[0.001, 0.02]	0.0061	Prevents policy degradation
CartPole	Training episodes	—	100,000	Maximum state coverage
CartPole	Discretization bins	[3,3,6,8] – [5,5,12,16]	[3,3,8,12]	864 states; finer θ , $\dot{\theta}$ bins
<i>Model-Based Methods (VI & PI)</i>				
Shared	Discount factor γ	[0.95, 1.00]	0.99	Maximizes long-term optimality
Shared	Convergence threshold θ	[10^{-6} , 10^{-3}]	10^{-4}	Tight accuracy; fast convergence
Shared	Max iterations	[500, 2,000]	1,000	Safety limit (never exceeded)
Shared	Evaluation episodes	—	100	Policy quality assessment
CartPole	Discretization bins	[3,3,6,8] – [6,6,14,16]	[4,4,10,12]	1,920 states; best return before slowdown
CartPole	MC samples per (s, a)	[10, 100]	50	Low transition variance; reasonable compute
<i>State Space Bounds (CartPole Discretization)</i>				
CartPole	Cart position x	—	[−2.4, 2.4]	Track boundary limits
CartPole	Cart velocity $\dot{x}$	—	[−3.0, 3.0]	Clamped for stability
CartPole	Pole angle θ	—	[−0.209, 0.209] rad	± 12 termination threshold
CartPole	Angular velocity $\dot{\theta}$	—	[−3.5, 3.5]	Clamped for bin coverage

less sample-efficient than model-based planning but is far more practical when the true dynamics are unknown, expensive, or change over time.

SARSA: State-Action-Reward-State-Action(SARSA) is a model-free, on-policy reinforcement learning algorithm that learns action values from experienced transitions. It computes an update target using the Q-value of the next state, occasionally utilizing the state-action Q-value of an explored action. The following describes the Bellman equation for finding the Q-function $Q(s, a)$.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

Q-Learning: Q-Learning is a model-free temporal-difference algorithm that approximates the optimal action-value function by choosing actions under the assumption that every subsequent greedy action will be the most optimal. The key mechanism that differentiates Q-Learning as an off-policy technique is that the agent may choose not to follow the highest rewarding ϵ -greedy action. The Q-value is always updated toward this maximum possible future reward, separating the behavior policy from the greedy policy and encouraging riskier exploration.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

Q-Learning fundamentally computes cumulative state-action rewards using the temporal-difference error, measuring the difference between the current Q-value estimate and the discounted value earned from choosing the next action with the maximum Q-value. Notice how it replaces the Q-value of the succeeding action with the objectively best action, regardless of what action is actually taken next. This is the principal divergence between the two model-free algorithms.

Model-Based Comparison: Value Iteration and Policy Iteration are classified as model-based dynamic programming algorithms that exploit a complete environmental model using a fragmented function architecture, preventing redundant recomputation. They are both equipped with all state transition probabilities and expected rewards, calculating all 280 Blackjack states via explicit probabilities and CartPole’s 1,920 discretized states via Monte Carlo sampling with fifty rollouts per state-action pair. Given their exhaustive search, the VI and PI algorithms are guaranteed to find the optimal policy for a finite MDP.

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V_k(s')]$$

From an arbitrary initial value function, Value Iteration performs repeated synchronous sweeps over the entire state space in order to solve the Bellman optimality equation. Initialized at zero, each iteration performs a one-step look-ahead to calculate the value of the next state until the maximum Bellman residual, or the value difference between iterations, is below a specified threshold.

$$\pi_{k+1}(s) = \operatorname{argmax}_a \sum_{s'} s', rp(s', r | s, a) [r + \gamma V^{\pi_k}(s')]$$

Policy Iteration alternates between two states: policy evaluation and policy improvement. It first solves the exact value function of the current policy via the linear Bellman expectation system, then creates a better policy through greedy updates, repeating both steps until convergence. Effectively, it repeatedly calculates the state-value function for the current policy and the expected total reward from each state until the policy converges.

Hyperparameter Tuning

The four algorithms rely on multiple parameters that influence state-action value estimation, each with distinct scales and sensitivities. To ensure reliable evaluation, each method requires moderate hyperparameter tuning to encourage stable

convergence. This report employs a three-stage tuning process for each algorithm, progressively refining the search space while simultaneously increasing episode counts for complexity-aware tuning. Initial values are generated using a lightweight coarse random search across broad parameter grids to explore large deviations from industry-standard default values. Successive halving then selects the strongest candidates and constructs finer ranges evaluated with moderate episode counts. In the final stage, the three highest-performing candidates undergo local refinement, and the resulting narrowed configurations are executed across all seeds at full production to ensure statistical robustness.

TABLE III
EXPERIMENTAL CONFIGURATION

Setting	Blackjack	CartPole
State space size	280 (discrete)	864–1,920 (discretized)
Action space	2 (hit/stand)	2 (left/right)
Episodes (model-free)	50,000	100,000
Seeds (SARSA/Q-Learning)	40	40
Seeds (VI/PI)	5	5
Solved threshold	Break-even (≥ 0)	195 steps/episode

Architectural Framework

The experiment framework is driven by a single Python entry point that loads a YAML configuration specifying the environments, all four algorithms with their hyperparameters, seed sets, runtime limits, and parallelization options. At startup, the runner initializes each environment using gymnasium and constructs the corresponding agent implementations, leveraging Numpy [4] and Pandas [5]. Environment-agent pairs follow a linear execution schedule, while a multiprocessing dispatcher iterates through seed batches. SARSA and Q-Learning are executed over forty seeds for stability assessment, whereas Value Iteration and Policy Iteration use five seeds due to their model-based, deterministic nature. All seeds remain fixed to ensure reproducibility.

Per-episode metrics are logged to CSV files, experiment metadata to JSON, and numerical tables to NPY arrays. After training, model-free methods evaluate their greedy policies over one hundred episodes, capturing means and standard deviations. Presentable tables, curves, and heatmaps are finally generated using matplotlib [6] and seaborn [7].

RESULTS AND ANALYSIS

After all production runs, the aggregated results are subjected to statistical analyzes quantifying differences in Value Iteration, Policy Iteration, SARSA, and Q-Learning across the discrete Blackjack MDP and the continuous CartPole MDP. Our evaluations focus on each agent’s final policy quality, measured by mean return and its stability through inter-quartile ranges, alongside computational complexity reflecting few episodes or iterations required for convergence.

Experimental Findings

Blackjack: Model-based methods achieve fifty percent stronger returns than model-free methods, aligning closely with historically observed casino probabilities. All four algorithms still produce negative mean returns, indicating that even after millions of hands the player should expect roughly a five percent loss. This outcome stems from the house edge created by the player acting before the dealer and the uncertainty of the dealer’s hidden card.

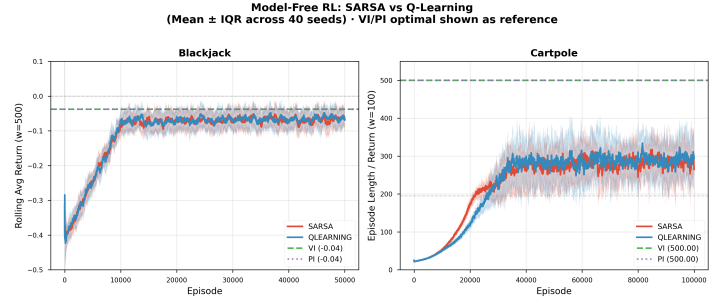


Fig. 2. Learning curves for SARSA and Q-Learning (40 seeds). Solid lines show mean return; shaded regions indicate IQR. (Top) Blackjack converges around episode 6,000. (Bottom) CartPole exceeds the 195-step threshold (dashed line).

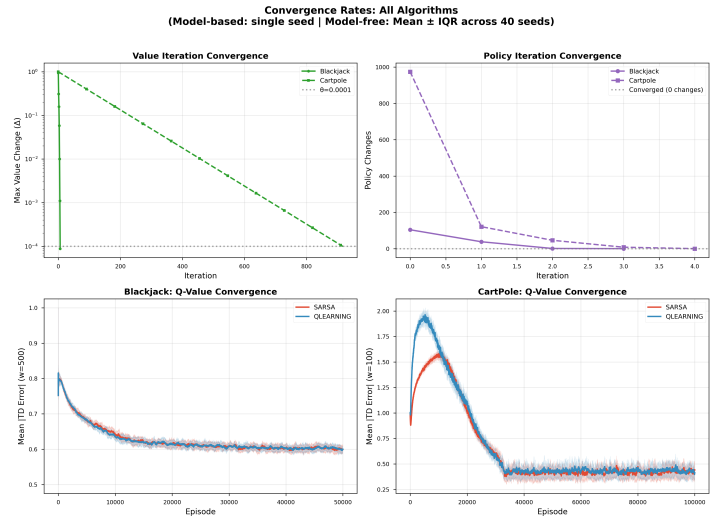


Fig. 3. Convergence diagnostics. VI requires 7 iterations on Blackjack and 918 on CartPole; PI converges in 3 and 5 outer steps respectively. Dashed line indicates threshold $\theta = 10^{-4}$.

TABLE IV
VALUE ITERATION VS POLICY ITERATION CONVERGENCE

Algorithm	Environment	Mean Return	Outer Iterations	Evaluation Sweeps	Wall-Clock (s)
VI	Blackjack	−0.048	7	—	0.039
PI	Blackjack	−0.048	3	23	0.047
VI	CartPole	16.73	918	—	23.47
PI	CartPole	52.97	5	1,844	8.75

TABLE V
SARSA VS Q-LEARNING PERFORMANCE (40 SEEDS)

Algorithm	Environment	Mean Return	Std	IQR	Wall-Clock
SARSA	Blackjack	−0.103	0.004	0.006	86.7
Q-Learning	Blackjack	−0.102	0.004	0.007	91.1
SARSA	CartPole	229.4	22.6	36.7	201.5
Q-Learning	CartPole	228.8	18.3	13.5	177.5

The stochastic nature of nondeterministic card draws appears in the model-free policy confidence curves, with action entropy almost doubling before convergence. With explicit transition probabilities, VI and PI converge within only a few iterations, directly exploiting known environment dynamics in a small observable state space. Meanwhile, the competing algorithms require thousands of episodes before converging to a weaker return. The heatmaps further show SARSA and Q-Learning producing noisier value estimates despite a near-identical policy, revealing susceptibility to greedy updates influenced by occasional outlying experiences and maximization bias on rare blackjacks [1]. These patterns reinforce our earlier claim that Blackjack’s stochastic structure benefits model-based planning more than model-free approximation.

CartPole: In contrast to Blackjack, SARSA and Q-Learning dominate in this environment, producing substantially higher returns and exceeding the solved threshold. The model-based algorithms perform poorly despite having full knowledge of the transition dynamics and even after discretizing the state space into finer bins. VI and PI expend thousands of sweeps on unreachable or low-relevance states, reducing emphasis on the near-balanced regions that matter most for control. As anticipated earlier, the model-free methods concentrate on states as they are encountered, allocating most updates to those that drive reward. They exhibit faster convergence than VI and PI by avoiding unnecessary iterations, supporting our claim and demonstrating a clever remedy to the curse of dimensionality. Conversely, the model-free algorithms show lower variability within CartPole, likely due to approximation errors introduced by discretization and longer episodes amplifying compounding inaccuracies, whereas the randomness in Blackjack tends to average out over thousands of short hands.

Value Iteration and Policy Iteration

In Blackjack, VI and PI converge after only seven and three outer iterations respectively, producing nearly identical mean returns and similar wall-clock times. With full dynamics and a small state space, both methods converge quickly enough that their differences are effectively negligible. Both VI and PI converged to identical greedy policies on Blackjack, differing by zero state-action pairs. CartPole shows a clear separation: Value Iteration requires about nine hundred full sweeps to meet the threshold, while Policy Iteration converges in five policy-improvement steps. PI finishes in roughly one-third of the time, hiding nearly two thousand inexpensive policy-evaluation updates. Under the same discretization, PI also achieves higher returns, indicating stronger tolerance to Monte-Carlo noise. When the value function is only approximated through discretization, the greedy improvement step is more reliable than VI’s incremental updates. Overall, state-space size appears to be the key factor driving the performance gap between VI and PI.

On-Policy Versus Off-Policy

As with the model-based methods, SARSA and Q-Learning display almost the same mean return likely from the environmental variance dwarfing their theoretical differences. They both converge between six and eight thousand episodes with minimal differences between IQR bands and similar variance. The off-policy algorithm shows a slightly noisier heatmap, hinting at its exploratory tendencies to hit more on higher, riskier summations.

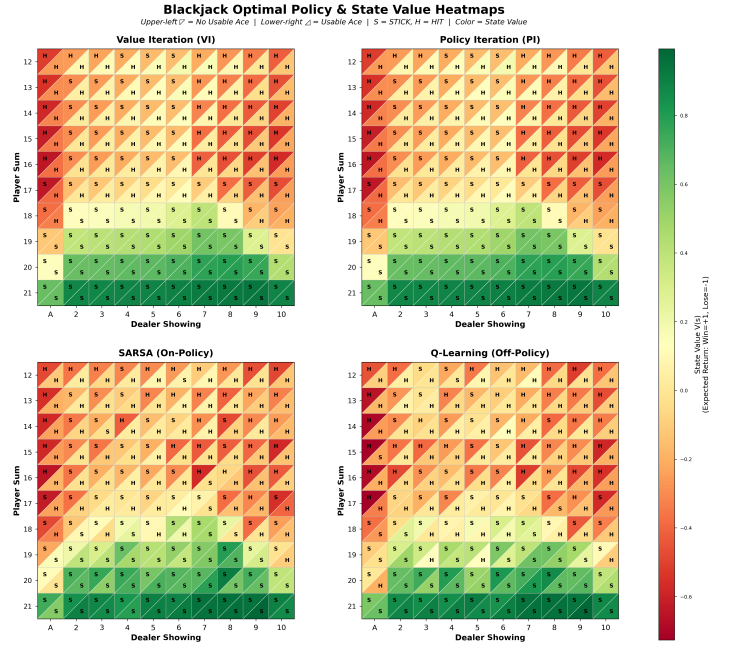


Fig. 4. Blackjack policy heatmaps showing optimal actions (hit/stand) across player sum and dealer card combinations. VI and PI converge to identical policies; SARSA and Q-Learning approximate similar structure.

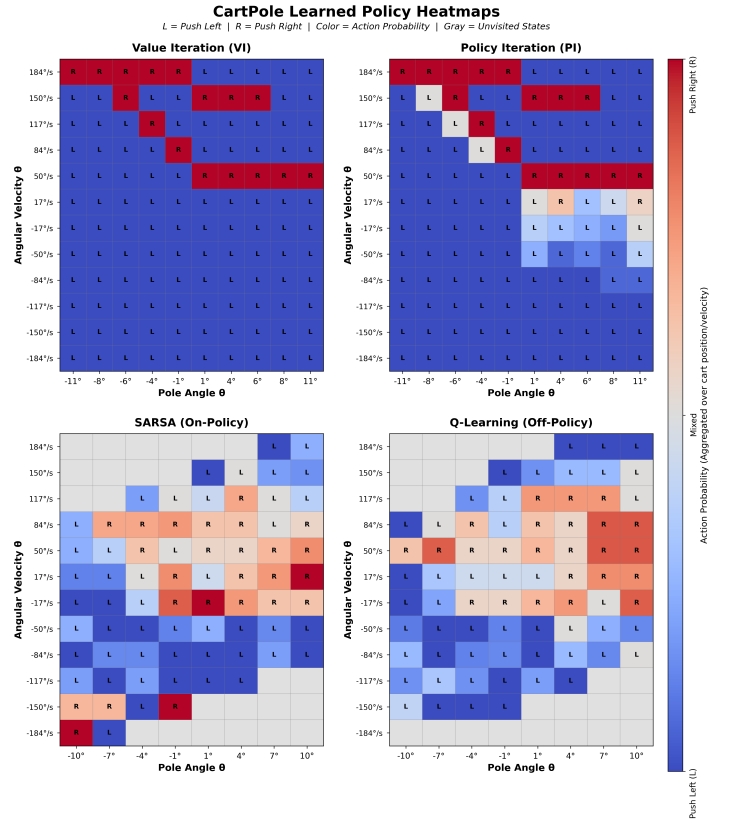


Fig. 5. Learned CartPole control strategy across discretized pole angle (θ) and angular velocity ($\dot{\theta}$). Actions cluster around the balanced region, demonstrating effective state-space coverage.

Regardless, the delayed rewards seem to dominate the learning signal, with subtle greediness hidden in environmental noise.

In CartPole, both methods achieve almost identical final returns, though several secondary metrics differ. SARSA reaches the solvable threshold sooner, likely accepting policies that are good enough as it follows its behavior policy’s trajectory. The smoother gradient within the heatmap signals Q-Learning’s exploration smoothing aggressive actions across the state space, while SARSA exploits actions it knows well. Q-Learning, somewhat unexpectedly, appears more stable with tighter variance and has a lower wall-clock time, though its max-operator can still overestimate returns when rare, high-reward transitions inflate the target [2].

The Q-function’s exploratory nature occasionally overestimates future returns, with rare high rewards inflating the max term and falling prey to its greedy policy. Although not implemented in code for simplicity, an instant Blackjack twenty-one can reaffirm unconventional and dangerous moves. Bellman’s equation takes advantage of CartPole’s physics-based determinism, reaching more states than SARSA while pushing the pole more aggressively towards the center.

TABLE VI
ALGORITHM PERFORMANCE SUMMARY BY ENVIRONMENT

Algorithm	Blackjack		CartPole	
	Return	Time (s)	Return	Time (s)
VI	−0.048	0.04	16.7	23.5
PI	−0.048	0.05	53.0	8.7
SARSA	−0.103	86.7	229.4	201.5
Q-Learning	−0.102	91.1	228.8	177.5

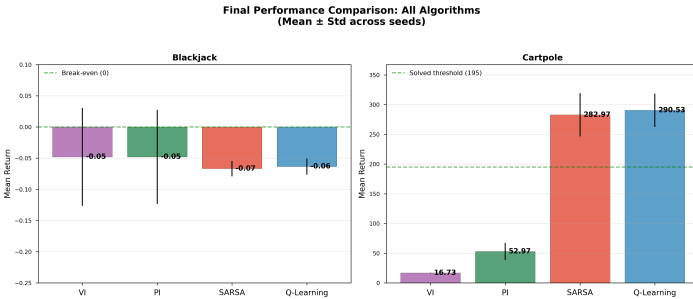


Fig. 6. Algorithm performance summary. Model-based methods achieve superior returns on Blackjack (−0.048 vs −0.102); model-free methods dominate Cart-Pole (229 vs 53 mean steps). Error bars show IQR.

CONCLUSION

This study iteratively validated the core alternatives between model-based and model-free algorithms for two distinct Markov Decision Problems. We analyzed individual algorithms, how they differed, and why we saw the results we did.

Key Findings

In the discrete, stochastic Blackjack environment, Value Iteration and Policy Iteration outperform the model-free methods by exploiting the fully known transition dynamics and traversing the small state space quickly and efficiently, staying consistent with our earlier claims. The explicit draw probabilities allow both model-based algorithms to avoid any guess-and-check behavior, few iterations, and stable convergence [1]. VI and PI converged

to nearly identical greedy policies across all 280 states, achieving only a five percent house edge.

The CartPole environment highlights the strengths of SARSA and Q-Learning in continuous state spaces. As anticipated, the enormous state space forces the model-based methods to rely on coarse discretizations, blurring their global knowledge and leading to weaker performance [1]. Q-Learning appears slightly more stable than SARSA, likely due to its max operator inflating value estimates through optimistic overestimation.

Off-policy updates can overvalue future states, especially regarding rare Blackjack winning hands, yet the results remained nearly identical to SARSA due to the heavy stochastic averaging. In CartPole, however, Q-Learning’s exploratory nature allows it to reach a broader set of transition states instead of being constrained to the specific trajectories encountered by SARSA, contributing to its stronger performance in this environment.

Final Takeaway

Overall, we have seen how state-space size plays a dominant role in reinforcement learning when applied to Markov Decision Problems. Large state-action spaces force model-based methods to perform exhaustive sweeps over exploding domains while offering outlier averaging and minimizing model-free approximation error in stochastic environments. Small state-spaces promote near-instant convergence with VI and PI, but when coupled with stochastic transitions, they generate additional noise for model-free methods to navigate through. Finally, we witnessed how off-policy algorithms explore riskier states that average transition strength across broader deterministic landscapes, while on-policy methods prioritize updating the few transitions that lower variance and promote higher return within stochastic environments.

Limitations

Hyperparameter search is shown to be very computationally expensive, requiring several full-scale, 2.5-hour production executions per parameter. Clear issues resulting from the continuous state space of CartPole proved difficult for model-based algorithms, exposing major weaknesses. The house edge intrinsic to Blackjack also misrepresented the full capabilities of all four methods, appearing less effective after analysis.

Tool Use Statement

Artificial intelligence was used in the planning, coding, debugging, formatting, and error checking stages of this project. GitHub’s Copilot [8], leveraging OpenAI’s ChatGPT [9], Anthropic’s Sonnet [10], and xAI’s Grok [11] were used to brainstorm report and software designs, generate functions (i.e. helpers, plotters, readme.md), and bugfix code. Microsoft Word’s embedded tool [12] helped fix spelling and grammar, and Google’s Gemini [13] made internet searches and documentation referencing more efficient.

REFERENCES

- [1] E. Ginzburg-Ganz, I. Segev, A. Balabanov *et al.*, “Reinforcement learning model-based and model-free paradigms for optimal control problems in power systems,” *Energies*, vol. 17, no. 21, p. 5307, 2024.
- [2] H. van Seijen, H. van Hasselt, S. Whiteson, and M. Wiering, “A theoretical and empirical analysis of expected SARSA,” *Journal of Machine Learning Research*, vol. 11, pp. 745–770, 2010.
- [3] M. Towers, J. K. Terry *et al.*, “Gymnasium,” <https://gymnasium.farama.org>, 2023.
- [4] C. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [5] W. McKinney, “Data structures for statistical computing in Python,” pp. 56–61, 2010.
- [6] J. Hunter, “Matplotlib: A 2d graphics environment,” *Computing in Science Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [7] M. L. Waskom, “seaborn: statistical data visualization,” *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021. [Online]. Available: <https://doi.org/10.21105/joss.03021>
- [8] “Github copilot,” <https://copilot.github.com/>, 2021, gitHub, Powered by OpenAI models.
- [9] “Chatgpt,” <https://chat.openai.com/>, OpenAI, 2023.
- [10] “Claude 3.5 sonnet,” <https://www.anthropic.com/claude>, Anthropic, 2024.
- [11] “Grok-1,” <https://github.com/xai-org/grok-1>, xAI, 2024.
- [12] “Microsoft editor,” <https://www.microsoft.com/en-us/microsoft-365/word/microsoft-editor>, Microsoft, 2024.
- [13] G. Team *et al.*, “Gemini: a family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2023.